NukMiris v0.8.1 — Independent Greenfield Architecture Challenge

Deliverable 5 of 5 — Final Recommendation

Independent architect submission · 26 October 2026 · Design only. References: Deliverable 1 (requirements assessment: F-n, T-n, S-n), Deliverable 2 (architecture: §), Deliverable 3 (ADR-n), Deliverable 4 (reliability and security challenge).

1. Recommendation

Build NukMiris as a **TypeScript modular monolith, deployed as two container roles (`web` and `worker`) over one managed, highly available PostgreSQL**, with Google Calendar as the truth for appointments and Postgres as the truth for execution history.

The design rests on five ideas that do most of the work:

1. **One transaction decides internal truth.** Accepted input, state change, queued replies and new operations commit together.
2. **External effects are bracketed.** A durable intent is recorded before every provider call and a verified outcome after it. Unknown outcomes are *uncertain*, never assumed.
3. **Retry only what is idempotent by construction.** Calendar creates use client-assigned IDs; WhatsApp sends are never retried automatically.
4. **Isolation is structural.** Tenant comes from the authenticated channel and is enforced by composite keys and row-level security, not by remembering to filter.
5. **Recovery is a normal code path,** drilled regularly, including restore after external side effects.

2. Why this architecture is appropriate

Your priority	How the design serves it
1. Reliability and correctness	Transactional inbox and outbox; fenced leases; reservation constraint; outcome model with a submission marker; sentinels

Your priority	How the design serves it
2. Security and tenant isolation	L1–L6 isolation layers; envelope-encrypted credentials bound to tenant; signature over raw bytes; opaque server-side offers; audited operator access
3. Maintainability	Pure domain core; hexagonal layering enforced in CI; one language, one database, one deploy unit
4. Testability and recovery	Pure reducer and engines; provider simulators with fault injection; crash-point harness; restore drill scripted and automated
5. Scalability and extensibility	Stateless processes scale out; Postgres has 10× headroom over the proposed envelope; multi-staff and push notifications are additive
6. Operational readiness	SLIs, alerts, review queue, runbooks, canary, private admin CLI
7. Performance and cost	No broker, no Kubernetes, small managed footprint; Calendar reads are lazy and bounded

The central point is that the hardest requirements in the baseline (I04–I09, I12, O03) are all about *what happens between two systems that cannot share a transaction*. A design that puts the inbox, state, ledger, outbox, schedule and arbiter in one transactional store, and makes every provider call restartable, addresses that directly and with the fewest moving parts.

3. Major strengths

- **Duplicate-proof by construction.** Duplicate webhooks, double taps and retried creates collapse on unique keys and deterministic identifiers instead of on timing luck.
- **Honest about uncertainty.** The customer is never told success for an unverified write, and nothing unknown is silently retried.
- **The database arbitrates concurrency.** Any number of instances is safe, because no correctness property depends on a process-local lock.
- **Recovery is designed, not hoped for.** The submission marker separates “never sent” from “maybe sent”; Calendar properties let events be rediscovered after a restore; a write-once archive recovers inbound input and withdrawals.

- **Small operational surface.** Two services and one database, plus managed KMS, secrets, object storage and telemetry.
- **Testable where it matters.** The dangerous windows are named crash points, exercised repeatedly, and acceptance is traced to requirement IDs rather than counted.

4. Weaknesses and trade-offs

1. **Up-front engineering cost.** Simulators, a crash harness and generated isolation tests are real work before the first customer. This is the main price of the design.
2. **Dependence on provider behaviours I could not verify.** Idempotent insert (S-1), the opaque callback echo (S-6) and several others are assumptions. If they fail, more uncertain cases need an operator.
3. **A bespoke scheduler on Postgres.** I own a small piece of scheduling code (leases, fences, claims). It is simple but must be exactly right, and it has no built-in visibility tooling.
4. **Privileged database features add complexity.** Row-level security, `SECURITY DEFINER` functions and an exclusion constraint demand careful review and carry pooling and planner subtleties (S-10).
5. **A single primary database.** Multi-AZ failover is automatic, but a regional outage means downtime up to the restore objective.
6. **Some risks are detected rather than prevented.** Manual or third-party double booking (U-EXTERNAL) and duplicated ordinary replies after a restore are bounded, not eliminated.
7. **Human handoff has no human behind it** (F-11). The architecture records and exposes it; the product still needs a process.
8. **More machinery than the POC.** A team accustomed to a single process and a local database will feel this.

5. Likely operational complexity

Area	What the team actually runs	Burden
Compute	Two container services (≥2 instances each)	Low

Area	What the team actually runs	Burden
Data	One managed Postgres with HA and PITR; migrations (expand then contract)	Medium: the single most important component
Security material	KMS keys, secrets manager, rotation procedures	Medium, mostly procedural
Provider relationships	Google OAuth verification; Meta business verification; per-business template approval and health	**Medium to high and ongoing;** it is a business process, not an engineering one
Observability	Telemetry backend, about a dozen paging alerts, a review queue, a canary business	Medium; needs tuning in the first months
Routine	Weekly restore drill; monthly credential, template and quota audit; time-zone data refresh	Low if automated, high if skipped
On-call	One rotation; page-worthy conditions are the §15.2 “Page” rows	Realistic for two to three engineers if alerts are disciplined

The largest operational load is likely to come from **provider standing and onboarding** (templates, number verification, consent evidence), not from the infrastructure.

6. What must be demonstrated before production

Gate	Evidence required
G1 Provider assumptions	Spikes S-1 to S-12 passed, or each fallback explicitly accepted by the owner
G2 Correctness under failure	Crash-point matrix, concurrency suite and all sentinels green in CI, including *fail before commit* and *succeed then lose the response* for every Calendar operation

Gate	Evidence required
G3 Isolation	Generated tenant-leakage suite green; role-privilege review of every `SECURITY DEFINER` function; an external security review or penetration test
G4 Recovery	A restore drill following §16.3 with zero lost withdrawals, zero duplicate reminders, all orphans surfaced; measured RPO and RTO against T-4
G5 Capacity	A soak and load test shaped like 10× T-1: webhook acknowledgement p99, claim latency, database headroom, Calendar quota use
G6 Provider standing	Google OAuth consent verified for production; Meta business verification; templates approved for pilot businesses; no refresh-token expiry surprises
G7 Live behaviour	At least two weeks on the production canary with zero sentinel violations and measured provider latency used to confirm the settle window and receipt-wait
G8 Operations	A game day exercising credential failure, Google outage and database failover with the real runbooks
G9 Owner and legal decisions	F-01 to F-05, F-12, F-13, the U-* items and T-1 to T-7 decided and recorded; privacy notice, retention and consent-evidence process agreed
G10 Pilot	Three to five friendly businesses under enhanced monitoring before general availability

7. Is it realistic for a small team and an early commercial product?

Yes, with discipline and an honest sequence; no, if the goal is a customer-facing launch in a few weeks.

Suggested sequence.

1. **Spikes (about two to three weeks).** Resolve S-1 to S-12 first. They decide whether ADR-006 and ADR-010 stand.

2. **Walking skeleton.** Webhook to inbox to reducer to outbox to send, running on simulators, with the crash harness from day one.
3. **Booking and management.** Reservations, the saga, lookup, move and cancel.
4. **Reminders and compliance.** Occurrences, preflight, permission evidence, templates, attendance.
5. **Hardening.** Isolation suite, restore drill, game day, security review.
6. **Pilot.**

Planning range (a judgement, not data). I would expect roughly four to six engineer-months to reach booking and management on the skeleton, and a further three to four for reminders, compliance and hardening, after the spike phase. Replace this with a real estimate once the spikes are done.

Minimum rigor line. If time forces cuts, these are the things I would defer and the things I would not.

Can be deferred (add before or soon after pilot)	Must not be cut
Polished dashboards and alert tuning	The submission marker and attempt ledger
Second-approver workflow for risky operator actions	The reservation constraint with fencing
Push notification channels	Row-level security and composite keys
Automated nightly restore in CI (keep the manual drill)	The monotonic receipt merge
Cross-region copies	Credential encryption bound to tenant
Fine-grained load testing beyond the 10× check	The crash-point suite and sentinels

The strongest argument against my design is that this is heavy for a product that has not yet found its market. My answer is that most of the weight is tables and discipline, not infrastructure, and that the expensive properties (idempotency, isolation, recoverable restore) are far cheaper to build in than to retrofit. Where that answer is weakest is the simulator and drill work, which is why I list it first among the trade-offs.

8. Decisions needed from the owner

- **Product and commercial:** F-01 (capacity one per Calendar), F-02 (WhatsApp account ownership and billing), F-04 (one Calendar, one business), F-05 (hold policy for uncertain sends), F-08 (no proactive result after the window), F-11 (human handoff process).
- **Interpretation:** F-03 (is a same-ID insert retry “not blind”?), U-EXTERNAL (policy for detected overlaps), U-REPLAY (customer wording).
- **Policy:** U-NAME, U-COMMAND, U-DST, U-ACTIVATION, U-GENERIC as proposed in Deliverable 1 §5.
- **Legal and operational:** F-12 (jurisdiction, retention, residency, erasure), U-OPS, and the numeric targets T-1 to T-7.

9. What would change my mind

- **S-1 fails** (no client-assigned event IDs): uncertain creates revert to read-only proof plus operator resolution; expect a higher operator load and reconsider the manual-recovery tooling investment.
- **S-6 fails** (no callback echo): more uncertain sends; strengthen operator tooling and shorten reminder windows to limit exposure.
- **S-10 fails** (constraint or RLS behaviour unsuitable on the chosen managed Postgres): use advisory locks with a unique slot index, and keep isolation in the application layer plus composite keys.
- **Claim contention or Calendar poll cost grows faster than expected:** introduce a broker for fan-out or push channels as a hint.
- **Multi-staff capacity or physical tenant isolation becomes a customer requirement:** revisit ADR-008 and ADR-012.
- **A stricter availability target than T-2 to T-4:** revisit ADR-016 for multi-region.

10. Changes I made while attacking my own design

Two changes came out of the challenge exercise and are reflected in the documents.

- **Operator command journal.** Restore scenario analysis showed that operator mutations (consent evidence, configuration, rotations) would be lost after the restore point. They are now journaled to the write-once archive and replayed before quarantine ends (Deliverable 2 §16.3; Deliverable 4 scenario 10).

- **Submission marker.** Crash analysis showed that without a marker written just before each provider call, every crash would cost a full settle window. A separate marker now distinguishes “never sent” from “maybe sent” (Deliverable 2 §8.3–8.4).

11. Closing statement

I am confident in the *shape*: one transactional store, bracketed external effects, idempotency only where it is real, structural isolation and a rehearsed recovery. I am less certain about several *parameters* (settle window, receipt-wait, polling intervals, targets T-1 to T-7) and about two provider behaviours (S-1, S-6). The spikes in Deliverable 1 §7 exist to retire exactly those doubts before any commitment is made.

This completes the independent architecture proposal. In line with the brief, I have not read documents 02–05 of the repository, have not modified any application code, schema, configuration or provider asset, and have stopped here.